

Functional Requirements

Retrieve Based on CDN models' CDN shld be able to retrieve data from Origin server

Request

- Content delivery from proxy server is made on user's req
- CDN proxy servers shld be able to respond to each user's req in this regard.

Delete

Possible to delete cached entries after certain period.

CDN

Delivers

Push Model

Origin Server shld be able to send data to CDN proxy servers.

Update

CDN shld be able to update the content within peer CDN proxy servers.

Search

CDN shld be able to execute a search against a user query for cached or otherwise stored content.

Non Functional Requirements

Performance

→ Minimize latency

Availability

→ Expected to be available at all times
→ Includes protection against DDoS

CDN

Scalability

→ User 1 ⇒ Scalable to scale horizon

Security

→ No SPOF
→ Handle massive traffic loads
→ Protection to hosted content from various attacks

Performance	Availability	Scalability	Reliability & Security
→ Proxy servers serve from RAM → CDN placed near to users for faster access → CDN can also be the provider of proxy servers. located in ISP's data center → Req Routing locates nearest proxy server	→ Deal with massive Traffic as distributed → Caching → If Proxy fails → other active → Load Balancers to distribute the users req to nearby proxy servers.	→ Brings content closer to user & removes the req. of high bandwidth → Add no. of replicas to seed in form of edge proxy servers. → Layered proxy archi. ⇒ scaled.	→ Maintenance Cycles Ensures no SPOF → Scrubber Servers for malicious traffic → Use heartbeat to monitor servers

#API Design

95

- 1) Retrieve [Proxy server to origin server]
Get method through /retrieve Content
retrieveContent(proxyserver-id, content-type, content-version, desc)
 ↳ unique ID of requesting proxy server.
 ↳ info obt requested content
 ↳ Category
 ↳ Type of Clients
 ↳ version of content
 ↳ current residing in proxy
 } Content Detail.
- 2) Deliver (Origin server to proxy server) /delivery Content
 ↳ Origin use this to deliver specified content, updated version,
 deliverContent(origin-id, server-list, content-type, content-version, desc)
 ↳ Origin server
 ↳ Server list to whom content has to deliver
 ↳ updated version of Content of origin server
- 3) Request (clients to proxy server) /request Content
 requestContent(user-id, content-type, desc)
 ↳ user ID of user who requested.
- 4) Search /search Content
 SearchContent(proxyserver-id, content-type, desc)
 (proxy to peer proxy)
 ↳ requesting proxy server.
- 5) update (proxy to peer proxy)
 updateContent(proxyserver-id, content-type, desc)
 ↳ John update Kone h.

96

* Content caching strategies in CDN

PUSH CDN

- Content automatically sent from origin to proxy server.
- Content delivery to proxy server is content providers responsibility
- Static content
- Origin decides which content to deliver.
- Content pushed to proxy servers in various loc acc. to locs popularity
- Maintains more replicas as static content is served to wide range of users for longer.

PULL CDN

- Pulls unavailable data from origin server when on users request.
- Proxy servers keep the file for a specified time & then removed from cache.
- CDN is itself resp to pull data from origin server & provide to users.
- Dynamic Content
- freq changing data & high traffic load.

* Dynamic Content Caching Optimization

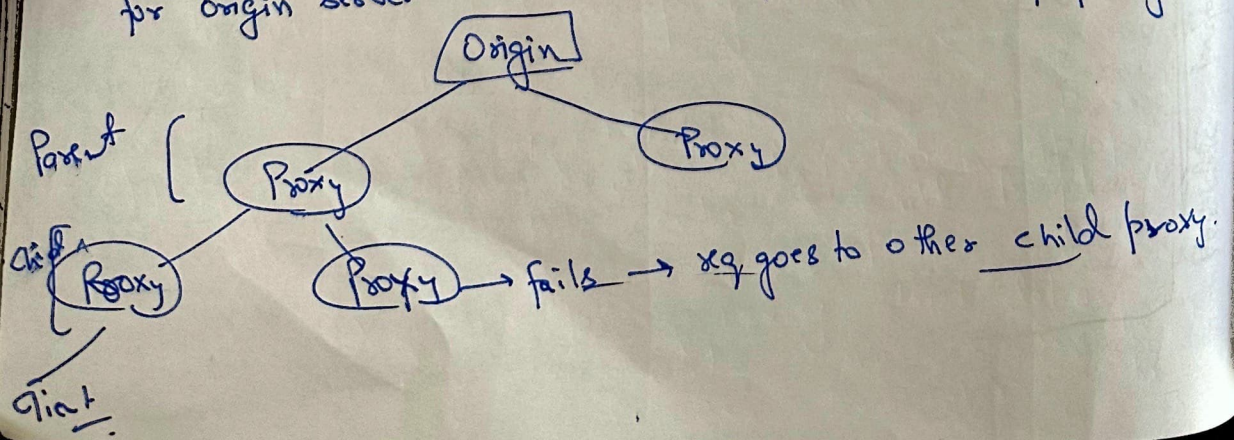
- If there are some scripts which need to run ⇒ run on proxy then on origin server.

Compression Techniques

- Reduce communication b/w origin & proxy server
- ESI ~~markup~~ long [when there is small change on Web page so fetching full page will bring lot of redundant data]
- Edge Side Includes
- specifies where content was changed so that rest of web page content can be cached

* Multitier CDN archi

- CDNs follow tree like structure to ease data distribution
- 1 or 2 tiers of proxy server



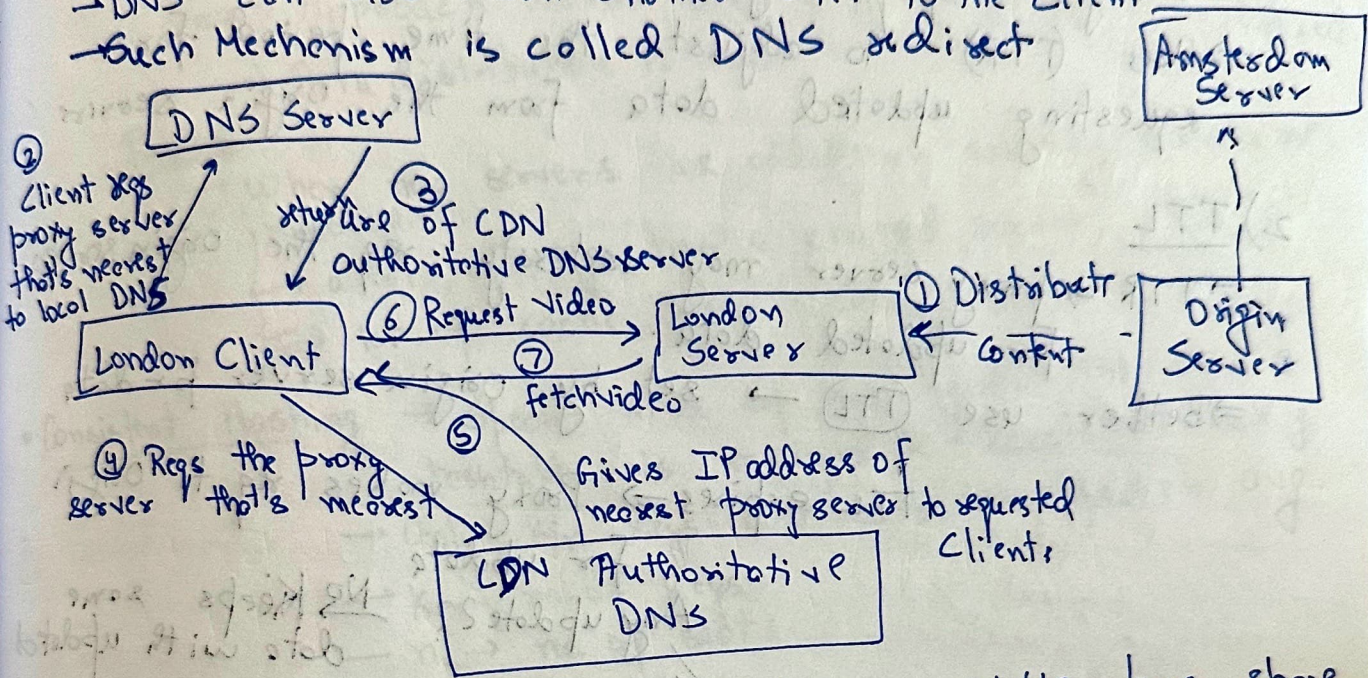
* Nearest Proxy server to fetch the data

Factors

- Network Distance: length of network path, capacity limit along network path $\downarrow$ & Bandwidth $\uparrow \Rightarrow$ Best
- Requests Loads: $\hookrightarrow$ Load a proxy server handles at any point of time.

1) DNS redirection

- $\rightarrow$ DNS can also return another URI to the client.
- $\rightarrow$ Such Mechanism is called DNS redirect



- ## 2) Anycast $\rightarrow$ All edge servers located in multiple locs share the same single IP address.
- $\rightarrow$ Employs BGP (Border Gateway Protocol) to route clients based on Internet's natural network flow.
 - $\rightarrow$ Direct client to nearest proxy server.

3) Client Multiplexing

- $\rightarrow$ It sends a list of candidate servers to client.
- $\rightarrow$ Client chooses one server from list to send req.
- $\rightarrow$ Inefficient bcz client doesn't know which is most suitable.

4) HTTP redirection

- $\rightarrow$ Client reqs content from origin server.
- $\rightarrow$ Origin server responds with on HTTP protocol to redirect the user via a URL of the content.

* Consistency in CDN

→ Users shld not consume stale data.

1) Periodic Polling

- Periodically pulls data from origin servers
- Risk of polling unnecessary when content changes infrequently.
- Uses (TTR) to adjust the time period for requesting updated data from the origin servers

2) TTL

→ TTR ⇒ proxy server may uselessly req. the origin server for updated data.

⇒ Better use TTL → set by origin server for data

TTL expires ⇒ Proxy makes req. to origin for update

Any update??

No Keeps same data with update

Yes updates data

TTL

3) Leases

→ Origin server grants lease to data sent to proxy server.

Time interval when origin server agrees to notify proxy server if there is any change in data.

→ Lease ends ⇒ Proxy shld send a msg for lease renewal.

→ Reduces msgs exchanged b/w proxy & origin server.

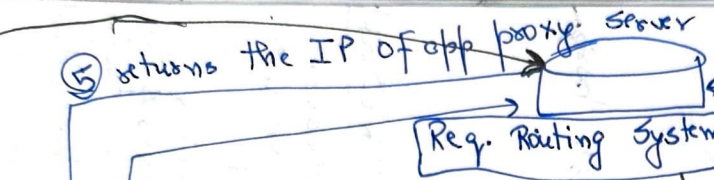
→ Lease duration can be optimized dynamically acc. to the observed load on proxy servers.

* When to Place CDN

Off premises → Placing CDN proxy servers in ISP's network suitable for global reach, fluctuating traffic patterns.

On Premises → Small Data center that could be placed near major IXP's

→ suitable for orgs having strict data privacy requirements, those serving local audience, or those who need to integrate deeply with existing



① Origin Server provide URI names space delegation of all obj's cached in CDN to routing system.

④ Request for suitable proxy server uses various routing mechanisms to route to nearest proxy server

Feed Back

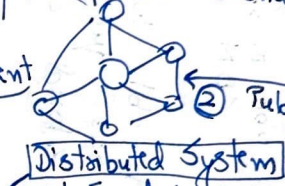
→ helps req routing sym to find nearest proxy server
→ Info abt cached content for requesting client its proxy server

Clients

⑧

Edge Proxy Server

③ Distribute Content



② Publishes content to Dist Sys

Origin Server

⑥

Scrubber Server

⑦

- Separate Good traffic from malicious.
- Protect from DDoS
- Used only when an attack is detected
- Traffic is scrubbed/cleaned & routed to Edge Proxy Server.

Periodically acc. info forwards

forwards

Periodic Info to Origin Server

Management System

→ In Case proxy Server does not have data

⇒ We can also use hierarchal Proxy servers
→ Miss ⇒ Hit Passen Proxy Server

ORIGIN SERVER

- Serves any unavailable data at CDN to clients
- Use app stores to keep content & other metadata

Periodic info of stats & detail of Content.